

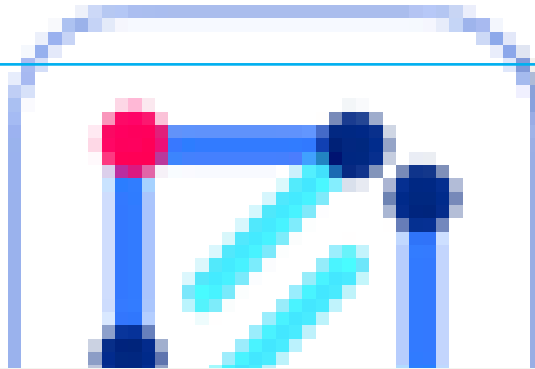


<u>Name</u>	<i><u>Muddassir Ali Siddiqui</u></i>
<u>Instructor</u>	<i><u>Sir Junaid</u></i>
<u>Date</u>	<i><u>23rd Sep 2025</u></i>

1. In-Lab Tasks:

i. Task 1:

Lesson Tutorial:



Exercise 1: Matching Characters

Task	Text	
Match	abcdefg	✓
Match	abcde	✓
Match	abc	✓

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 1½: Matching Digits

Task	Text	
Match	abc123xyz	✓
Match	define "123"	✓
Match	var g = 123;	✓

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 1½: Matching Digits

Task	Text	
Match	abc123xyz	✓
Match	define "123"	✓
Match	var g = 123;	✓

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Scripting

Exercise 1½: Matching Digits

Task	Text	
Match	abc123xyz	✓
Match	define "123"	✓
Match	var g = 123;	✓

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 2: Matching With Wildcards

Task	Text	
Match	cat.	✓
Match	896.	✓
Match	?=+.	✓
Skip	abc1	✗

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 2: Matching With Wildcards

Task	Text	
Match	cat.	✓
Match	896.	✓
Match	?=+.	✓
Skip	abc1	

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Scripting

Exercise 3: Matching Characters

Task	Text
------	------

Match	can
-------	-----



Match	man
-------	-----



Match	fan
-------	-----



Skip	dan
------	-----

Skip	ran
------	-----

Skip	pan
------	-----

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 4: Excluding Characters

Task	Text
------	------

Match	hog
-------	-----



Match	dog
-------	-----



Skip	bog
------	-----

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 5: Matching Character Ranges

Task	Text
------	------

Match	Ana
-------	-----



Match	Bob
-------	-----



Match	Cpc
-------	-----



Skip	aax
------	-----

Skip	bby
------	-----

Skip	ccz
------	-----

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Scripting

Exercise 6: Matching Repeated Characters

Task	Text	
Match	wa zzzz up	✓
Match	wa zzz up	✓
Skip	wazup	

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 6: Matching Repeated Characters

Task	Text	
Match	w azzzzz up	✓
Match	w azzzz up	✓
Skip	wazup	

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 7: Matching Repeated Characters

Task	Text	
Match	a aa abcc	✓
Match	a abbbb c	✓
Match	a acc	✓
Skip	a	

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Scripting

Exercise 8: Matching Optional Characters

Task	Text	
Match	1 file found?	✓
Match	2 files found?	✓
Match	24 files found?	✓
Skip	No files found.	

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 9: Matching Whitespaces

Task	Text	
Match	1. abc	✓
Match	2. abc	✓
Match	3. abc	✓
Skip	4.abc	

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 10: Matching Lines

Task	Text	
Match	Mission: successful	✓
Skip	Last Mission: unsuccessful	
Skip	Next Mission: successful upon capture of target	

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Scripting

Exercise 11: Matching Groups

Task	Text	Capture Groups
Capture	file_record_transcript.pdf	file_record_transcript ✓
Capture	file_07241999.pdf	file_07241999 ✓
Skip	testfile_fake.pdf.tmp	

Continue ›

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 13: Matching Nested Groups

Task	Text	Capture Groups
Capture	1280x720	1280 720 ✓
Capture	1920x1600	1920 1600 ✓
Capture	1024x768	1024 768 ✓

Continue ›

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 14: Matching Conditional Text

Task	Text	
Match	I love cats	✓
Match	I love dogs	✓
Skip	I love logs	
Skip	I love cogs	

Continue ›

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 15: Matching Other Special Characters

Task	Text
------	------

Match	The quick brown fox jumps over the lazy dog.
-------	--



Match	There were 614 instances of students getting 90.0% or above.
-------	--



Match	The FCC had to censor the network for saying &##*@!.
-------	--

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 15: Matching Other Special Characters

Task	Text
------	------

Match	The quick brown fox jumps over the lazy dog.
-------	--



Match	There were 614 instances of students getting 90.0% or above.
-------	--



Match	The FCC had to censor the network for saying &##*@!.
-------	--

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 15: Matching Other Special Characters

Task	Text	
Match	The quick brown fox jumps over the lazy dog.	✓
Match	There were 614 instances of students getting 90.0% or above.	✓
Match	The FCC had to censor the network for saying &\$#@!.	✓

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Lesson X: Infinity and beyond!



You've finished the tutorial!

Scripting

Practice Problems:

Exercise 1: Matching Numbers

Task	Text	
Match	3.14529	✓
Match	-255.34	✓
Match	128	✓
Match	1.9e10	✓
Match	123,340.00	✓
Skip	720p	

`^[+-]?(?:\d{1,3}(?:,\d{3})*|\d+)(?:\.\d+)?(?:[eE][`

Continue ›

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 2: Matching Phone Numbers

Task	Text	Capture Groups	
Capture	415-555-1234	415	✓
Capture	650-555-2345	650	✓
Capture	(416)555-3456	416	✓
Capture	202 555 4567	202	✓
Capture	4035555678	403	✓
Capture	1 416 555 9292	416	✓

`^(?:1[\s-]*)?(?:\((\d{3})\)|[\s-]*\d{3}[\s-]*\d{`

Continue ›

Solve the above task to continue on to the next problem, or read the [Solution](#).

Scripting

Exercise 3: Matching Emails

Task	Text	Capture Groups	
Capture	tom@hogwarts.com	tom	✓
Capture	tom.riddle@hogwarts.com	tom.riddle	✓
Capture	tom.riddle+regexone@hogwarts.com	tom.riddle	✓
Capture	tom@hogwarts.eu.com	tom	✓
Capture	potter@hogwarts.com	potter	✓
Capture	harry@hogwarts.com	harry	✓
Capture	hermione+regexone@hogwarts.com	hermione	✓

`^([A-Za-z0-9. -]+)(?:\+[^@]+)?@[\\w. -]+\.[A-Za-z]{2}`

Continue ›

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 4: Capturing HTML Tags

Task	Text	Capture Groups	
Capture	<a>This is a link	a	✓
Capture	Link	a	✓
Capture	<div class='test_style'>Test</div>	div	✓
Capture	Hello world	div	✓

`<([A-Za-z][A-Za-z0-9]*)\b[>]*>.*?</\1>`

Continue ›

Solve the above task to continue on to the next problem, or read the [Solution](#).

Scripting

Exercise 5: Capturing Filename Data

Task	Text	Capture Groups
Skip	.bash_profile	
Skip	workspace.doc	
Capture	img0912.jpg	img0912 jpg ✓
Capture	updated_img0912.png	updated_img0912 ✓ png
Skip	documentation.html	
Capture	favicon.gif	favicon gif ✓
Skip	img0912.jpg.tmp	
Skip	access.lock	

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Exercise 6: Matching Lines

Task	Text	Capture Groups
Capture	The quick brown fox...	The quick brown fox... ✓
Capture	jumps over the lazy dog.	jumps over the lazy dog. ✓

[Continue ›](#)

Solve the above task to continue on to the next problem, or read the [Solution](#).

Scripting

Exercise 7: Extracting Data From Log Entries

Task	Text	Capture Groups
Skip	W/dalvikvm(1553): threadid=1: uncaught exception	
Skip	E/(1553): FATAL EXCEPTION: main	
Skip	E/(1553): java.lang.StringIndexOutOfBoundsException	
Capture	E/(1553): at widget.List.makeView(ListView.java:1727)	makeView ListView.java1727 ✓
Capture	E/(1553): at widget.List.fillDown(ListView.java:652)	fillDown ListView.java652 ✓
Capture	E/(1553): at widget.List.fillFrom(ListView.java:709)	fillFrom ListView.java709 ✓

at\s+\S+\.(\\w+)\((\\w+\\.java):(\\d+)\)

Continue ›

Solve the above task to continue on to the next problem, or read the [Solution](#).

Scripting

Exercise 8: Extracting Data From URLs

Task	Text	Capture Groups
Capture	<code>ftp://file_server.com:21/top_secret/life_changing_plans.pdf</code>	ftp file_server.com 21 ✓
Capture	<code>https://regexone.com/lesson/introduction#section</code>	https regexone.com ✓
Capture	<code>file://localhost:4040/zip_file</code>	file localhost 4040 ✓
Capture	<code>https://s3cur3-server.com:9999/</code>	https s3cur3-server.com 9999 ✓
Capture	<code>market://search/angry%20birds</code>	market search ✓
^(\\w+):/(\\w\\. -)+(?::(\\d+))?		Continue ›

Solve the above task to continue on to the next problem, or read the [Solution](#).



RegexOne

Learn Regular Expressions with simple, interactive exercises.



Interactive Tutorial



References & More

Problem X: Infinity and beyond!

Congrats on getting through all the problems! To learn how to use regular expressions in some common programming languages, continue on to the references in the menu in the header above. Otherwise, we hope you can start applying regular expressions in your day to day work!

If you find the lessons and problems useful, please consider [donating \(\\$4\)](#) to support RegexOne. Your contribution will help keep the servers running and allow us to add even more material in the future.

Have questions or suggestions on how to improve the site? Feel free to get in touch via Twitter ([@regexone](#)) or email (regexone@gmail.com)!

ii. Task 2a:

```
[cc@ncdc-0063 src]$ bash task_2_a.sh 80000058
Method 1 - Using grep and sed:
addi

Method 2 - Using awk:
addi

Method 3 - Using grep with pattern:
addi
[cc@ncdc-0063 src]$
```

iii. Task 2b:

```
[cc@ncdc-0063 src]$ bash task_2_b.sh
```

```
Total qualifying words: 703
```

```
Top 3 last-two-letter endings:
```

```
54 nd
```

```
48 a
```

```
44 at
```

```
Unique two-letter endings: 81
```

```
Two-letter combinations that do NOT occur:
```

```
grep: SOURCE_FILE.txt: No such file or directory
```

```
aa,ab,ac,ad,ae,af,ag,ah,ai,aj,ak,al,am,an,ao,ap,aq,ar,as,at,au,av,aw,ax,ay,az,ba,bb,bc,bd,be,bf,bg,bh,bi,bj,bk,bl,bm,bn,bo,bp,bq,br,bs,bt,bu,bv,bw,bx,by,bz,ca,cb,cc,cd,ce,cf,cg,ch,ci,cj,ck,cl,cm,cn,co,cp,cq,cr,cs,ct,cu,cv,cw,cx,cy,cz,da,db,dc,dd,de,df,dg,dh,di,dj,dk,dl,dm,dn,do,dp,dq,dr,ds,dt,du,dv,dw,dx,dy,dz,ea,eb,ec,ed,ee,ef,eg,eh,ei,ej,ek,el,em,en,eo,ep,eq,er,es,et,eu,ev,ew,ex,ey,ez,fa,fb,fc,fd,fe,ff,fg,fh,fi,fj,fk,fl,fm,fn,fo,fp,fq,fr,fs,ft,fu,fv,fw,fx,fy,fz,ga,gb,gc,gd,ge,gf,gg,gh,gi,gj,gk,gl,gm,gn,go,gp,gq,gr,gs,gt,gu,gv,gw,gx,gy,gz,ha,hb,hc,hd,he,hf,hg,hh,hi,hj,hk,hl,hm,hn,ho,hp,hq,hr,hs,ht,hu,hv,hw,hx,hy,hz,ia,ib,ic,id,ie,if,ig,ih,ii,ij,ik,il,im,in,io,ip,iq,ir,is,it,iu,iv,iw,ix,iy,iz,ja,jb,jc,jd,je,jf,jg,jh,ji,jj,jk,jl,jm,jn,jo,jp,jq,jr,js,jt,ju,jv,jw,jx,jy,jz,ka,kb,kc,kd,ke,kf,kg,kh,ki,kj,kl,km,kn,ko,kp,kq,kr,ks,kt,ku,kv,kw,kx,ky,kz,la,lb,lc,ld,le,lf,lg,lh,li,lj,lk,ll,lm,ln,lo,lp,lq,lr,ls,lt,lu,lv,lw,lx,ly,lz,ma,mb,mc,md,me,mf,mg,mh,mi,mj,mk,ml,mm,mn,mo,mp,mq,mr,ms,mt,mu,mv,mw,mx,my,mz,na,nb,nc,nd,ne,nf,ng,nh,ni,nj,nk,nl,nm,nn,no,np,nq,nr,ns,nt,nu,nv,nw,nx,ny,nz,oa,ob,oc,od,oe,of,og,oh,oi,oj,ok,ol,om,on,oo,op,oq,or,os,ot,ou,ov,ow,ox,oy,oz,pa,pb,pc,pd,pe,pf,pg,ph,pi,pj,pk,pl,pm,pn,po,pp,pq,pr,ps,pt,pu,pv,pw,px,py,pz,qa,qb,qc,qd,qe,qf,qg,qh,qi,qj,qk,ql,qm,qn,qo,qp,qq,qr,qs,qt,qu,qv,qw,qx,qy,qz,ra,rb,rc,rd,re,rf,rg,rh,ri,rj,rk,rl,rm,rn,ro,rp,rq,rr,rs,rt,ru,rv,rw,rx,ry,rz,sa,sb,sc,sd,se,sf,sg,sh,si,sj,sk,sl,sm,sn,so,sp,sq,sr,ss,st,su,sv,sw,sx,sy,sz,ta,tb,tc,td,te,tf,tg,th,ti,tj,tk,tl,tm,tn,to,tp,tq,tr,ts,tt,tu,tv,tw,tx,ty,tz,ua,ub,uc,ud,ue,uf,ug,uh,ui,uj,uk,ul,um,un,uo,up,uq,ur,us,ut,uu,uv,uw,ux,uy,uz,va,vb,vc,vd,ve,vf,vg,vh,vi,vj,vk,vl,vm,vn,vo,vp,vq,vr,vs,vt,vu,vv,vw,vx,vy,vz,wa,wb,wc,wd,we,wf,wg,wh,wi,wj,wk,wl,wm,wn,wo,wp,wq,wr,ws,wt,wu,wv,ww,wx,wy,wz,xa,xb,xc,xd,xg,xe,xf,xg,xh,xi,xj,xk,xl,xm,xn,xo,xp,xq,xr,xs,xt,xu,xv,xw,xx,xy,xz,ya,yb,yc,yd,ye,yf,yg,yh,yi,yj,yk,yl,ym,yn,yo,yp,yq,yr,ys,yt,yu,yv,yw,yx,yy,yz,za,zb,zc,zd,ze,zf,zg,zh,zi,zj,zk,zl,zm,zn,zo,zp,zq,zr,zs,zt,zu,zv,zw,zx,zy,zz
```

```
[cc@ncdc-0063 src]$
```

```
[cc@ncdc-0063 files]$ grep -oE '\w+' article.txt | tr '[:upper:]' '[:lower:]' |  
grep 'a' | grep -v 'i$' | wc -l  
703
```

```
[cc@ncdc-0063 files]$ grep -oE '\w+' article.txt | tr '[:upper:]' '[:lower:]' |  
grep 'a' | grep -v 'i$' | awk '{print substr($0, length($0)-1)}' | sort | uniq -  
c | sort -nr | head -3  
54 nd  
48 a  
44 at
```

```
[cc@ncdc-0063 files]$ grep -oE '\w+' article.txt | tr '[:upper:]' '[:lower:]' |  
grep 'a' | grep -v 'i$' | awk 'length($0) > 1 { print substr($0, length($0)-1) }'  
' | sort -u | wc -l  
81  
[cc@ncdc-0063 files]$
```

```
[cc@ncdc-0063 files]$ comm -23 \  
> <(for x in {a..z}; do for y in {a..z}; do echo "$x$y"; done; done | sort) \  
> <(grep -oE '\w+' article.txt \  
> | tr '[:upper:]' '[:lower:]' \  
> | grep 'a' \  
> | grep -v 'i$' \  
> | awk 'length($0) > 1 {print substr($0, length($0)-1)}' \  
> | sort -u) \  
> | paste -sd, -  
aa,ab,ac,ae,af,ah,ai,aj,ak,ao,aq,au,av,aw,ax,az,ba,bb,bc,bd,be,bf,bg,bh,bi,bj,bk,bl,bm,bn,bo,bp,bq,br,bs,bt,bu,bv,bw,bx,by,bz,ca,cb,cc,cd,cf,cg,ci,cj,ck,cl,cm,cn,co,cp,cq,cr,cu,cv,cw,cx,cy,cz,da,db,dc,de,df,dg,dh,di,dj,dk,dl,dm,dn,do,dp,dq,dr,ds,dt,du,dv,dw,dx,dy,ea,eb,ec,ee,ef,eg,eh,ei,ej,ek,el,em,en,ep,eq,eu,ev,ew,ex,ez,fa,fb,fc,fd,fe,ff,fg,fh,fi,fj,fk,fl,fm,fn,fo,fp,fq,fr,fs,ft,fu,fv,fw,fx,fy,fz,ga,gb,gc,gd,gf,gg,gh,gi,gj,gk,gl,gm,go,gp,gq,gr,gs,gt,gu,gv,gw,gx,gz,ha,hb,hc,hd,he,hf,hg,hh,hi,hj,hk,hl,hn,ho,hp,hq,hr,hs,ht,hu,hv,hw,hx,hy,hz,ib,ie,if,ig,ih,ii,ij,ik,il,im,in,iq,is,iu,iv,iw,ix,iy,iz,ja,jb,jc,jd,je,jf,jg,jh,ji,jj,jk,jl,jm,jn,jo,jp,jq,jr,js,jt,ju,jv,jw,jx,jy,jz,ka,kb,kc,kd,kf,kg,kh,ki,kj,kl,km,kn,ko,kp,kq,kr,kt,ku,kv,kw,kx,ky,kz,lb,lc,ld,lg,lh,li,lj,lk,lm,ln,lo,lp,lq,lr,lt,lu,lv,lw,lx,lz,ma,mb,mc,md,mf,mg,mh,mi,mj,mk,ml,mm,mn,mo,mp,mq,mr,mt,mu,mv,mw,mx,my,mz,na,nb,nc,nf,nh,ni,nj,nl,nm,nn,no,np,nq,nr,nu,nv,nw,nx,nz,oa,ob,oc,od,oe,of,og,oh,oi,oj,ol,om,oo,op,oq,os,ot,ou,ov,ow,ox,oy,oz,pa,pb,pc,pd,pe,pf,pg,ph,pi,pj,pk,pl,pm,pn,po,pp,pq,pr,ps,pt,pu,pv,pw,px,py,pz,qa,qb,qc,qd,qe,qf,qg,qh,qi,qj,qk,ql,qm,qn,qo,qp,qq,qr,qs,qt,qu,qv,qw,qx,qy,qz,ra,rb,rc,rd,rf,rg,rh,ri,rj,rk,rl,ro,rp,rq,rr,ru,rv,rw,rx,ry,sa,sb,sc,sd,sf,sg,sh,si,sj,sk,sl,sm,sn,so,sq,sr,ss,sv,sw,sx,sy,sz,tb,tc,td,tf,tg,th,ti,tj,tk,tl,tm,tn,to,tp,tr,tt,tu,tv,tw,tx,ty,ua,ub,uc,ud,ue,uf,ug,uh,ui,uj,uk,um,un,uo,up,uq,ur,us,ut,uv,uw,ux,uy,uz,va,vb,vc,vd,ve,vf,vg,vh,vi,vj,vk,vl,vm,vn,vo,vp,vq,vr,vs,vt,vu,vv,vw,vx,vz,wa,wb,wc,wd,we,wf,wg,wh,wi,wj,wk,wl,wm,wn,wo,wp,wq,wr,ws,wt,wu,wv,ww,wx,wy,wz,xa,xb,xc,xd,xe,xf,xg,xh,xi,xj,xk,xl,xm,xn,xo,xp,xq,xr,xs,xt,xu,xv,xw,xx,xy,xz,ya,yb,yc,yd,ye,yf,yg,yh,yi,yj,yk,yl,ym,yn,yo,yp,yq,yr,yt,yu,yv,yw,yx,yy,yz,za,zb,zc,zd,zf,zg,zh,zi,zj,zk,zl,zm,zn,zo,zp,zq,zr,zs,zt,zu,zv,zw,zx,zy,zz
```

2. Critical Analysis:

In Lab 28, I explored practical text-processing and scripting skills on Linux. First, I completed the interactive regex tutorial at regexone.com, gaining hands-on experience and saved a screenshot of the finished exercises. Next, I created a Bash script that accepts a program-counter value and extracts only the corresponding assembly instruction from `core.txt`, using tools such as `grep`, `awk`, and `sed`. For the second part, I processed `article.txt` to identify every word containing at least one “a” but not ending with “i”.

I counted all such words, found the three most common last-two-letter endings, and calculated how many unique two-letter combinations appear. Finally, I generated a comma-separated list of all two-letter combinations that never occur in those words. This lab reinforced regular-expression concepts, case-insensitive filtering, and the power of Unix text utilities, while improving my ability to chain commands into efficient analysis pipelines.